Informe Tecnico: Hosting Capacity - Analisis de Capacidad de Alojamiento de Generacion Distribuida

---

Tabla de Contenidos

1. Descripcion General del Proyecto
2. Flujo de Funcionamiento
3. Estructura del Repositorio
4. Funciones Principales y Firmas
5. Tipos de Entradas y Salidas
6. Modelos de Datos
7. Algoritmos Clave
8. Dependencias del Sistema
9. Estado Actual del Codigo
10. Recomendaciones para Migracion a Web
11. Diseno de Endpoints REST API

---

1. Descripcion General del Proyecto

Este proyecto implementa una plataforma de analisis de redes de distribucion electrica orientada a calcular la **Capacidad de Alojamiento de Generacion Distribuida** (Hosting Capacity). El sistema determina la maxima potencia de generacion distribuida (GD) que puede conectarse a cada barra del circuito IEEE 13 Nodos sin producir violaciones de los limites normativos de voltaje, corriente o potencia.

El motor de simulacion es **OpenDSS**, un simulador de sistemas de potencia electrica de codigo abierto desarrollado por EPRI. La interfaz grafica utiliza **Dash** (framework web de Python basado en Flask y Plotly), con integracion opcional a **Power BI** mediante exportacion a Excel y JSON.

Proposito principal: Calcular, visualizar y analizar la maxima generacion distribuida admisible por nodo del sistema de distribucion, evaluando el impacto en perfiles de voltaje, corrientes de linea y perdidas del sistema.

---

2. Flujo de Funcionamiento

2.1 Flujo Principal del Sistema

```
┌──────────────────────────────────────────────────────────────┐
│                     INICIO DE APLICACION                     │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│            CARGA DEL CIRCUITO IEEE 13 NODOS                  │
│  - Ejecuta: Clear → Redirect IEEE13Nodeckt.dss → Solve       │
```

```
│  - Inicializa motor OpenDSS                                    │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼

┌──────────────────────────────────────────────────────────────┐
│           CALCULO DE ESTADO BASE (SIN GD)                      │
│  - Voltajes PU por barra y fase                                │
│  - Perdidas del sistema (kW, kvar)                             │
│  - Informacion de lineas (NormAmps, S_nominal)                 │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼

┌──────────────────────────────────────────────────────────────┐
│      BUSQUEDA BINARIA DE LIMITES DE HOSTING CAPACITY           │
│  Para cada combinacion barra-fase:                             │
│  - Rango: 0 kW a 1,500,000 kW                                  │
│  - Aplica GD, resuelve circuito, verifica violaciones          │
│  - Determina maximo kW sin violaciones                         │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼

┌──────────────────────────────────────────────────────────────┐
│              DASHBOARD INTERACTIVO (DASH)                      │
│  - Tabla de hosting capacity por barra/fase                    │
│  - Perfil de voltaje inicial                                   │
│  - Interfaz de aplicacion de GD manual                         │
└──────────────────────────────────────────────────────────────┘
                              │
                ┌─────────────┴─────────────┐
                ▼                           ▼

┌──────────────────────┐    ┌──────────────────────────────┐
│   USUARIO SELECCIONA │    │  RESULTADO AUTOMATICO        │
│   GD MANUAL          │    │  DE HOSTING CAPACITY         │
│  - Barra             │    │  - Tabla pivot barra x fase  │
│  - Fase              │    └──────────────────────────────┘
│  - kW                │
│  - Tipo (1F/2F/3F)   │
└──────────────────────┘
            │
            ▼

┌──────────────────────────────────────────────────────────────┐
│           SIMULACION CON GD APLICADA                           │
│  1. Reinicia circuito al estado base                           │
│  2. Crea generador (Generator.GD) con parametros dados         │
│  3. Resuelve el circuito                                       │
│  4. Detecta violaciones de voltaje (< 0.95 o > 1.05 PU)        │
│  5. Detecta violaciones de corriente (> NormAmps)              │
│  6. Detecta violaciones de potencia (> S_nominal)              │
│  7. Calcula perdidas antes y despues                           │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼

┌──────────────────────────────────────────────────────────────┐
│              VISUALIZACION DE RESULTADOS                       │
```

```
│  - Grafico comparativo de voltaje PU (con/sin GD)           │
│  - Tabla de violaciones de voltaje                           │
│  - Tabla de violaciones de corriente                         │
│  - Tabla de violaciones de potencia                          │
│  - Resumen de perdidas con/sin GD                            │
│  - Grafico de barras de perdidas totales                     │
└──────────────────────────────────────────────────────────────┘
```

2.2 Flujo de la Version Modular (con Carga de Archivos)

```
Pantalla Bienvenida
    │
    ├── Upload: archivo principal (.dss)      [Requerido]
    ├── Upload: linecodes (.dss)              [Opcional]
    └── Upload: coordenadas de barras (.csv)  [Opcional]
         │
         ▼
    Boton "Analizar Archivos"
         │
         ▼
    Decodificacion base64 → archivo temporal
         │
         ▼
    OpenDSSAnalyzer.ejecutar_analisis_completo()
         │
         ├── reiniciar_circuito()
         ├── obtener_perfil_voltajes()
         ├── obtener_loss_table()
         └── analizar_limites_gd()
         │
         ▼
    Almacenamiento en dcc.Store
         │
         ▼
    Redireccion a /analyze
         │
         ▼
    Layout de analisis con datos
```

2.3 Flujo del Modulo Power BI

```
Power BI invoca script Python
    │
    Parametros via JSON en argv[1]:
    { "barra": "634", "fase": 1, "potencia_kW": 50 }
    │
    ▼
ejecutar_simulacion(barra, fase, potencia_kW)
    │
```

```
    ├── Carga circuito base
    ├── Captura voltajes sin GD
    ├── Agrega generacion distribuida
    ├── Resuelve circuito
    ├── Captura voltajes con GD
    └── Detecta violaciones de voltaje
    │
    ▼
Exportacion a resultados_temp.xlsx
    - Hoja "Voltajes"
    - Hoja "Perdidas"
    - Hoja "Violaciones"
    │
    ▼
print(json.dumps(resultados))  → Power BI lee stdout
```

---

3. Estructura del Repositorio

```
Hosting-Capacity/
│
├── APP/
│   └── mi_app.py                          # Prototipo Streamlit
(descartado)
│
├── PROGRAMA DINAMICO/
│   └── dss_powerbi.py.py                  # Integracion con Power BI
│
├── OpenDSS/
│   ├── IEEETestCases/
│   │   └── 13Bus/                         # DIRECTORIO PRINCIPAL DE
TRABAJO
│   │       ├── IEEE13Nodeckt.dss          # Definicion del circuito IEEE
13 Nodos
│   │       ├── Simulacion_DASH_VFinal.py  # Aplicacion final (version
monolitica)
│   │       ├── Simulacion_DASH_VFinal_2.py
│   │       ├── Simulacion_DASH_V1.py      # Iteraciones de desarrollo
│   │       ├── Simulacion_DASH_V3.py
│   │       ├── Simulacion_DASH_V10.py
│   │       ├── Simulacion_DASH_V11.py
│   │       ├── dss_powerbi.py             # Copia del modulo Power BI
│   │       │
│   │       ├── Modular/                   # Arquitectura modular (mas
reciente)
│   │       │   ├── app.py                 # Punto de entrada Dash con
routing
│   │       │   ├── analysis_module.py     # Clase OpenDSSAnalyzer
│   │       │   ├── default_files.py       # Linecodes por defecto
│   │       │   ├── components/
│   │       │   │   ├── welcome.py         # Pantalla de bienvenida /
```

```
upload
│   │       │   │   ├── layout.py          # Layout de analisis
│   │       │   │   ├── callbacks.py       # Callbacks Dash
│   │       │   │   ├── voltage_profile.py # Componente perfil de voltaje
│   │       │   │   ├── gd_limits.py       # Componente limites GD
│   │       │   │   ├── gd_controls.py     # Componente controles GD
│   │       │   │   └── results.py         # Componente resultados
│   │       │   ├── opendss/
│   │       │   │   ├── dss_engine.py      # Wrapper del motor DSS
│   │       │   │   ├── analysis.py        # Funciones de analisis
│   │       │   │   └── gd_analysis.py     # Analisis especifico de GD
│   │       │   └── utils/
│   │       │       └── helpers.py         # Utilidades generales
│   │       │
│   │       ├── New/                       # Esqueleto de nueva
arquitectura
│   │       │   ├── app.py
│   │       │   └── src/
│   │       │       ├── callbacks.py
│   │       │       ├── layouts.py
│   │       │       └── core_dss.py
│   │       │
│   │       └── SS/                        # Variacion alternativa
│   │
│   ├── Examples/                          # Ejemplos oficiales de OpenDSS
│   ├── EPRITestCircuits/                  # Circuitos de prueba EPRI
│   ├── x64/                               # Binarios de OpenDSS 64-bit
│   └── Doc/                               # Documentacion de OpenDSS
│
├── 07abril.xlsx                           # Resultados de simulacion (7
de abril)
├── 12abril.xlsx                           # Resultados de simulacion (12
de abril)
├── DATOS DE ITERACIONES.xlsx              # Registro de iteraciones de
busqueda
├── ITERACIONES mil tablas copia.xlsx      # Copias de tablas de
iteraciones
│
├── DATOS-GRAFICOS 5-5-25.pbix            # Dashboard Power BI principal
├── DATOS-GRAFICOS.pbix
├── RESULTADOS.pbix
├── EJEMPLO.pbix
└── 12ABRIL.pbix
```

---

4. Funciones Principales y Firmas

4.1 Modulo: `Simulacion_DASH_VFinal.py` (Aplicacion Principal)

`carga_valida(total_power_kw: float) -> float`

Protege contra division por cero en calculos de porcentaje de perdidas.

- **Entrada:** valor de potencia total en kW (puede ser cero o negativo)
- **Salida:** valor absoluto del parametro, o `1e-3` si el valor es menor al umbral
- **Efecto lateral:** ninguno

`obtener_loss_table() -> Tuple[pd.DataFrame, Dict[str, str]]`

Calcula la tabla de perdidas por elemento del circuito activo.

- **Entrada:** ninguna (opera sobre el estado activo del motor OpenDSS)
- **Salida:**
  - `DataFrame` con columnas: `["Tipo", "Elemento", "kW Perdida", "% of Power", "kvar Perdida"]`
  - `Dict` con claves:
    - `"Perdidas Totales (kW)"`: string numerico
    - `"Perdidas Totales (kvar)"`: string numerico
    - `"Potencia Total de Carga (kW)"`: string numerico
- **Proceso:** itera sobre Lines, Transformers y Capacitors del circuito

---

`reiniciar_circuito() -> None`

Restaura el circuito al estado base IEEE 13 Nodos sin GD conectada.

- **Entrada:** ninguna
- **Salida:** ninguna
- **Efecto lateral:** ejecuta `Clear`, `Redirect IEEE13Nodeckt.dss`, `Solve` en el motor OpenDSS

Callback `actualizar_fases(barra: str) -> Tuple`

Callback Dash reactivo a la seleccion de barra.

- **Entrada:** nombre de la barra seleccionada (str)
- **Salida:** opciones del dropdown de fases, valor inicial del dropdown, texto informativo
- **Tipos de salida:** `List[Dict]`, `None`, `str`

Callback `actualizar_grafico(n_clicks, barra, fase_seleccionada, kw, tipo) -> Tuple`

Callback principal que ejecuta la simulacion con GD y retorna todas las visualizaciones.

- **Entradas:**
  - `n_clicks` (int): contador de clicks del boton
  - `barra` (str): nombre de la barra destino
  - `fase_seleccionada` (int | List[int]): fase o fases seleccionadas
  - `kw` (float): potencia de la GD en kW, rango 0-150,000
  - `tipo` (List[str]): lista con `"trifasica"`, `"bifasica"`, o vacia para monofasica
- **Salidas:**
  - Figura Plotly comparativa de voltajes

    - Mensaje de validacion (str)
    - Componente HTML con tablas de violaciones
    - Componente HTML con tabla de perdidas comparativa

4.2 Modulo: `PROGRAMA DINAMICO/dss_powerbi.py.py` (Integracion Power BI)

`obtener_barras() -> List[str]`

Carga el circuito base y retorna la lista de todas las barras.

- **Entrada:** ninguna
- **Salida:** lista de nombres de barras (strings)

`obtener_voltajes_por_fase() -> Dict[str, Dict[int, float]]`

Calcula los voltajes en por unidad de todas las barras y fases del circuito activo.

- **Entrada:** ninguna (estado activo del circuito)
- **Salida:** diccionario `{ "nombre_barra": { 1: voltaje_pu, 2: voltaje_pu, 3: voltaje_pu } }`
- **Nota:** fases no presentes en la barra son `None`; voltajes menores a `1e-3` se registran como `0.0`

`obtener_perdidas() -> float`

Retorna las perdidas activas totales del sistema.

- **Entrada:** ninguna
- **Salida:** perdidas en kW (float, redondeado a 4 decimales)

`agregar_generacion_distribuida(barra: str, potencia: float) -> None`

Agrega un generador (modelo PV) a la barra especificada.

- **Entradas:**
    - `barra` (str): nombre de la barra de conexion
    - `potencia` (float): potencia total a inyectar en kW
- **Logica de configuracion de fases:**
    - 3 fases disponibles: crea generador trifasico a 4.16 kV
    - 2 fases disponibles: crea dos generadores monofasicos a 2.4 kV con `potencia/2` cada uno
    - 1 fase disponible: crea un generador monofasico a 2.4 kV
- **Efecto lateral:** modifica el circuito activo en memoria

`ejecutar_simulacion(barra: str, fase: int, potencia_kW: float) -> Dict`

Funcion principal de simulacion para la integracion con Power BI.

- **Entradas:**
  - `barra` (str): nombre de la barra objetivo
  - `fase` (int): fase de referencia (1, 2 o 3)
  - `potencia_kW` (float): potencia de la GD en kW
- **Salida:** diccionario con estructura:

```
{
  "config": { "barra": str, "fase": int, "potencia_kW": float },
  "voltajes": [
    { "barra": str, "voltaje_sin_gd": float, "voltaje_con_gd": float
},
    ...
  ],
  "perdidas": { "sin_gd": float, "con_gd": float },
  "violaciones": [
    { "barra": str, "voltaje": float },
    ...
  ]
}
```

4.3 Clase: `OpenDSSAnalyzer` (Modulo Modular)

`__init__(self) -> None`

Inicializa el motor OpenDSS y todos los contenedores de datos internos.

- **Efecto lateral:** llama a `dss.DSS.Start(0)`, instancia referencias a `Text`, `ActiveCircuit`, `Solution`

`reiniciar_circuito(self, dss_content: str) -> None`

Carga un circuito DSS desde contenido en memoria.

- **Entrada:** `dss_content` (str): contenido completo del archivo `.dss`
- **Proceso:** crea archivo temporal, ejecuta `Clear` → `Compile` → `Solve`, elimina el archivo temporal
- **Efecto lateral:** actualiza `self.barras` con la lista de barras del circuito cargado

---

`obtener_perfil_voltajes(self) -> pd.DataFrame`

Calcula el perfil de voltajes en por unidad de todas las barras.

- **Entrada:** ninguna (usa estado interno del motor)
- **Salida:** `DataFrame` con columnas `["Barra.Fase", "VoltajePU"]`
- **Nota:** incluye entradas `None` para las combinaciones barra-fase no existentes en el circuito

---

`obtener_loss_table(self) -> Tuple[pd.DataFrame, Dict]`

Version robusta del calculo de perdidas, con manejo de errores por elemento.

- **Salida identica** a la funcion standalone, pero actualiza `self.resumen_sin_gd` como efecto lateral

---

`analizar_limites_gd(self) -> List[Dict]`

Version simplificada del calculo de hosting capacity (usa valor fijo de 1000 kW por barra-fase).

- **Nota:** esta version no ejecuta busqueda binaria para evitar bloqueos. La busqueda binaria completa esta implementada en `Simulacion_DASH_VFinal.py`.
- **Salida:** `List[{"Barra": str, "Fase": int, "Max GD sin violacion (kW)": float}]`

`ejecutar_analisis_completo(self, dss_content: str) -> Dict[str, Any]`

Orquesta la ejecucion completa del analisis del circuito.

- **Entrada:** `dss_content` (str): contenido del archivo `.dss`
- **Salida:**

```
{
  "voltajes_df": List[Dict],
  "limites_gd": List[Dict],
  "perdidas_sin_gd_df": List[Dict],
  "resumen_sin_gd": Dict,
  "barras": List[str],
  "barras_fases_disponibles": Dict[str, List[int]],
  "circuit_info": {
    "name": str,
    "num_buses": int,
    "num_elements": int,
    "converged": bool,
    "total_power_kw": float,
    "total_power_kvar": float
  },
  "original_dss_content": str
}
```

---

5. Tipos de Entradas y Salidas

5.1 Entradas del Sistema

Origen	Parametro	Tipo	Descripcion	Restricciones
Interfaz web	Barra	`string`	Nombre del nodo de conexion	Debe existir en el circuito

Origen	Parametro	Tipo	Descripcion	Restricciones
Interfaz web	Fase	`int` o `List[int]`	Fase(s) de conexion	1, 2 o 3; deben estar disponibles en la barra
Interfaz web	Potencia GD (kW)	`float`	Potencia activa de la generacion distribuida	0 a 150,000 kW
Interfaz web	Tipo de GD	`List[string]`	`"trifasica"`, `"bifasica"` o monofasica (vacio)	Debe ser compatible con las fases disponibles
Archivo	Circuito DSS	`string`	Contenido del archivo `.dss` (texto plano)	Formato OpenDSS valido
Archivo	Linecodes DSS	`string`	Definiciones de codigos de linea	Formato OpenDSS valido; opcional
Archivo	Coordenadas CSV	`string`	Coordenadas XY de barras	Formato CSV; opcional
Power BI / CLI	JSON en argv[1]	`string`	`{"barra": str, "fase": int, "potencia_kW": float}`	JSON valido

5.2 Salidas del Sistema

Componente	Salida	Tipo	Descripcion
Dashboard	Perfil de voltaje	Grafico Plotly (barras)	Voltaje PU por barra.fase para el estado base
Dashboard	Tabla hosting capacity	HTML Table	Pivot de maximo kW por barra y fase
Dashboard	Grafico comparativo de voltaje	Grafico Plotly	Voltajes antes y despues de aplicar GD con limites
Dashboard	Tabla de violaciones de voltaje	HTML Table	Barras.fases fuera del rango 0.95-1.05 PU
Dashboard	Tabla de violaciones de corriente	HTML Table	Lineas con corriente superior a NormAmps
Dashboard	Tabla de violaciones de potencia	HTML Table	Lineas con potencia aparente superior a S_nominal
Dashboard	Resumen de perdidas	Lista HTML + Grafico	Perdidas kW y kvar con y sin GD
Power BI	Excel (3 hojas)	`.xlsx`	Voltajes, Perdidas, Violaciones
Power BI	JSON en stdout	`string`	Resultado serializado de la simulacion

Componente	Salida	Tipo	Descripcion
API interna	`Dict` de analisis	Python dict	Resultado completo del analisis del circuito

5.3 Rangos y Restricciones de Valores

Variable	Unidad	Valor Minimo	Valor Maximo	Valor Aceptable
Voltaje	PU	0.0	cualquiera	0.95 a 1.05
Corriente de linea	Amperios	0.0	dependiente del circuito	<= NormAmps
Potencia aparente de linea	kVA	0.0	dependiente del circuito	<= S_nominal
Potencia GD (interfaz)	kW	0	150,000	cualquiera dentro del rango
Potencia GD (busqueda binaria)	kW	0	1,500,000	limite superior teorico
Perdidas totales	kW	0.0	sin limite superior definido	menor es mejor

6. Modelos de Datos

6.1 Perfil de Voltaje

```
{
  "Barra.Fase": "632.1",
  "VoltajePU": 1.021456
}
```

6.2 Elemento de Perdidas

```
{
  "Tipo": "Lines",
  "Elemento": "650632",
  "kW Perdida": 0.04521,
  "% of Power": 0.12,
  "kvar Perdida": 0.02134
}
```

6.3 Resumen de Perdidas del Sistema

```
{
  "Perdidas Totales (kW)": "1.23456",
  "Perdidas Totales (kvar)": "0.56789",
  "Potencia Total de Carga (kW)": "-3500.00"
}
```

6.4 Limite de Hosting Capacity por Barra-Fase

```
{
  "Barra": "634",
  "Fase": 1,
  "Max GD sin violacion (kW)": 87500
}
```

6.5 Violacion de Voltaje

```
{
  "Barra.Fase": "634.1",
  "Voltaje con GD (PU)": 1.0632
}
```

6.6 Violacion de Corriente

```
{
  "Linea": "650632",
  "Fase": 1,
  "Corriente (A)": 412.5,
  "Limite (A)": 400.0
}
```

6.7 Violacion de Potencia

```
{
  "Linea": "650632",
  "Fase": 2,
  "Potencia (kVA)": 2850.3,
  "Limite (kVA)": 2400.0
}
```

6.8 Resultado de Simulacion (Power BI)

```
{
  "config": {
    "barra": "634",
    "fase": 1,
    "potencia_kW": 500.0
  },
  "voltajes": [
    { "barra": "650", "voltaje_sin_gd": 1.0000, "voltaje_con_gd": 1.0012
},
    { "barra": "632", "voltaje_sin_gd": 1.0201, "voltaje_con_gd": 1.0215 }
  ],
  "perdidas": {
    "sin_gd": 87.3400,
    "con_gd": 75.2100
  },
  "violaciones": []
}
```

6.9 Informacion del Circuito

```
{
  "name": "IEEE13Nodeckt",
  "num_buses": 16,
  "num_elements": 58,
  "converged": true,
  "total_power_kw": -3466.1,
  "total_power_kvar": -1990.6
}
```

---

7. Algoritmos Clave

7.1 Busqueda Binaria para Hosting Capacity

Para cada combinacion (barra, fase), se ejecuta el siguiente algoritmo:

```
PARA cada barra en barras:
    PARA cada fase en nodos_disponibles(barra):

        low = 0
        high = 1,500,000
        best_kw = 0

        MIENTRAS low <= high:
            mid = (low + high) // 2

            reiniciar_circuito()
            agregar_GD(barra, fase, mid kW)
```

```
            solve_circuito()

            violacion_voltaje = verificar_voltajes()   // <0.95 o >1.05 PU
            violacion_corriente = verificar_corrientes() // >NormAmps
            violacion_potencia = verificar_potencias()   // >S_nominal

            SI no hay violaciones:
                best_kw = mid
                low = mid + 1
            SINO:
                high = mid - 1

        REGISTRAR: (barra, fase, best_kw)
```

Complejidad: O(B * F * log(1,500,000)) donde B es el numero de barras y F el numero de fases por barra. Para el IEEE 13 Nodos con ~35 combinaciones barra-fase, implica aproximadamente 735 soluciones del circuito.

7.2 Calculo de Voltaje en Por Unidad

```
V_pu = sqrt(V_real^2 + V_imag^2)
```

Donde `V_real` y `V_imag` son los componentes del voltaje fasor obtenidos de `ActiveBus.puVoltages`.

7.3 Calculo de Potencia Aparente de Linea

```
# Para lineas trifasicas:
S_real = sqrt(3) * kV_base * I_magnitud

# Para lineas monofasicas:
S_real = kV_base * I_magnitud
```

7.4 Porcentaje de Perdidas

```
porcentaje = (perdida_elemento_kW / potencia_total_carga_kW) * 100
```

---

8. Dependencias del Sistema

8.1 Dependencias Python

Paquete	Version Minima Recomendada	Uso en el Proyecto

Paquete	Version Minima Recomendada	Uso en el Proyecto
`dss` (opendssdirect.py)	0.8+	Motor de simulacion OpenDSS; acceso via `dss.DSS`
`opendssdirect`	0.8+	Version alternativa del cliente Python para OpenDSS
`dash`	2.14+	Framework web para el dashboard interactivo
`dash_bootstrap_components`	1.5+	Componentes Bootstrap para Dash (tema Flatly)
`plotly`	5.18+	Visualizaciones interactivas (graficos de barras, scatter)
`pandas`	2.0+	Manipulacion de DataFrames para resultados
`streamlit`	1.28+	Solo en `APP/mi_app.py` (no utilizado en version final)
`streamlit_echarts`	0.4+	Solo en `APP/mi_app.py` (no utilizado en version final)
`json`	stdlib	Serializacion/deserializacion de datos
`sys`	stdlib	Lectura de argumentos de linea de comandos (Power BI)
`tempfile`	stdlib	Creacion de archivos temporales para compilacion DSS
`os`	stdlib	Operaciones de sistema de archivos
`time`	stdlib	Medicion de tiempos de ejecucion
`traceback`	stdlib	Captura de trazas de error detalladas
`base64`	stdlib	Decodificacion de archivos subidos via Dash Upload
`typing`	stdlib	Anotaciones de tipos (`Dict`, `List`, `Tuple`, `Any`)

8.2 Dependencias del Sistema Operativo

Componente	Requisito	Descripcion
OpenDSS Engine	Version 9.6+ (x64)	Motor de simulacion; binarios en `OpenDSS/x64/`
Python	3.9+	Interprete Python de 64 bits

Componente	Requisito	Descripcion
Sistema Operativo	Windows (principal)	Los binarios de OpenDSS incluidos son para Windows x64

8.3 Dependencias de Datos

Archivo	Tipo	Requerido	Descripcion
`IEEE13Nodeckt.dss`	OpenDSS circuit	Si	Definicion completa del circuito IEEE 13 Nodos
Linecodes DSS	OpenDSS linecode	No (tiene defaults)	Matrices de impedancia de lineas
Coordenadas CSV	CSV	No	Posiciones XY de barras para mapa geografico

8.4 Archivo `requirements.txt` Recomendado

```
dss>=0.8.0
opendssdirect>=0.8.0
dash>=2.14.0
dash-bootstrap-components>=1.5.0
plotly>=5.18.0
pandas>=2.0.0
openpyxl>=3.1.0
```

---

9. Estado Actual del Codigo

9.1 Archivos en Estado de Produccion

Archivo	Estado
`Simulacion_DASH_VFinal.py`	Produccion; version monolitica funcional con busqueda binaria real
`PROGRAMA DINAMICO/dss_powerbi.py.py`	Produccion; integracion con Power BI
`Modular/analysis_module.py`	En desarrollo; busqueda binaria simplificada (limite fijo 1000 kW)
`Modular/app.py`	En desarrollo; arquitectura modular con soporte de carga de archivos

9.2 Limitaciones Identificadas

1. **Busqueda binaria en modulo Modular:** la clase `OpenDSSAnalyzer` usa un valor fijo de 1000 kW en lugar de ejecutar la busqueda binaria real. Esto es una simplificacion deliberada para evitar bloqueos

en el servidor web.

2. **Acoplamiento con rutas absolutas:** `Simulacion_DASH_VFinal.py` usa `"Redirect IEEE13Nodeckt.dss"` sin ruta absoluta; requiere ejecutarse desde el directorio `13Bus/`.
3. **Estado global del motor OpenDSS:** el circuito se inicializa como variable global al arrancar la aplicacion; en un entorno multi-usuario, el estado del motor puede corromperse.
4. **Sin autenticacion ni control de acceso:** la aplicacion Dash no implementa ningun mecanismo de autenticacion.
5. **Tiempo de arranque elevado:** la busqueda binaria completa para todos los nodos puede tomar varios minutos; esto bloquea el hilo principal de Flask/Dash.

---

10. Recomendaciones para Migracion a Web

10.1 Arquitectura Recomendada

Se recomienda separar completamente el motor de calculo del frontend, siguiendo una arquitectura cliente-servidor con API REST:

```
┌──────────────────────────────────────────────────────────────┐
│                      FRONTEND WEB                            │
│  React / Vue.js / Next.js                                    │
│  - Formularios de parametros                                 │
│  - Graficos con Plotly.js o Chart.js                         │
│  - Tablas de resultados                                      │
│  - Gestion de estado (Redux / Pinia)                         │
└──────────────────────────────────────────────────────────────┘
                          │
                          │ HTTP / WebSocket
                          ▼
┌──────────────────────────────────────────────────────────────┐
│                   BACKEND API REST                           │
│  FastAPI (Python)                                            │
│  - Endpoints REST                                            │
│  - Validacion de entrada (Pydantic)                          │
│  - Manejo de sesiones de circuito                            │
│  - Cola de tareas (Celery + Redis)                           │
└──────────────────────────────────────────────────────────────┘
          │                                  │
          ▼                                  ▼
┌──────────────────────┐          ┌──────────────────────────┐
│  Motor OpenDSS       │          │   Base de Datos          │
│  (proceso Python)    │          │   PostgreSQL / SQLite    │
│  opendssdirect       │          │   - Resultados cacheados │
└──────────────────────┘          │   - Historial de analisis │
                                  └──────────────────────────┘
```

10.2 Framework Backend Recomendado: FastAPI

FastAPI es la opcion recomendada por las siguientes razones:

- Soporte nativo de tipos Python (compatibilidad directa con los modelos de datos existentes)
- Generacion automatica de documentacion OpenAPI / Swagger
- Compatibilidad con Pydantic para validacion de entradas
- Soporte asincrono para operaciones largas (busqueda binaria)
- Ecosistema maduro para Python cientifico

10.3 Manejo de Operaciones de Larga Duracion

La busqueda binaria de hosting capacity puede tomar varios minutos. Se recomienda implementar un patron de trabajo asincrono:

```
POST /api/analysis/hosting-capacity
    → Retorna: { "task_id": "abc123" }

GET /api/analysis/tasks/{task_id}/status
    → Retorna: { "status": "running", "progress": 45 }

GET /api/analysis/tasks/{task_id}/result
    → Retorna: { "status": "completed", "data": {...} }
```

Herramientas recomendadas para la cola de tareas:

- **Celery** con broker **Redis** para tareas distribuidas
- **FastAPI BackgroundTasks** para casos simples sin distribucion

10.4 Manejo de Estado del Motor OpenDSS

El motor OpenDSS no es thread-safe. En un entorno web multi-usuario, se recomienda:

1. **Pool de workers:** cada proceso worker de Celery tiene su propia instancia de OpenDSS
2. **Serializacion de circuito:** pasar el contenido DSS como parametro (ya implementado en `OpenDSSAnalyzer`)
3. **Sin estado global:** nunca compartir la instancia del motor entre requests

---

11. Diseno de Endpoints REST API

A continuacion se presenta el diseno completo de la API REST para la migracion del sistema a web.

11.1 Endpoints de Gestion de Circuito

`POST /api/circuit/upload`

Sube y valida un archivo de circuito DSS.

Content-Type: `multipart/form-data`

Campos del formulario:

Campo	Tipo	Requerido	Descripcion
`main_dss`	File (.dss)	Si	Archivo principal del circuito
`linecodes_dss`	File (.dss)	No	Archivo de codigos de linea
`busxy_csv`	File (.csv)	No	Coordenadas de barras

Respuesta 200:

```
{
  "circuit_id": "ckt_7f3a21b",
  "circuit_info": {
    "name": "IEEE13Nodeckt",
    "num_buses": 16,
    "num_elements": 58,
    "converged": true,
    "total_power_kw": -3466.1,
    "total_power_kvar": -1990.6
  },
  "expires_at": "2025-04-08T10:00:00Z"
}
```

Respuesta 400:

```
{
  "error": "INVALID_FILE_FORMAT",
  "message": "El archivo no tiene formato OpenDSS valido",
  "detail": "Error en linea 45: sintaxis incorrecta"
}
```

`GET /api/circuit/{circuit_id}/info`

Retorna la informacion basica del circuito cargado.

Path parameters: `circuit_id` (string)

Respuesta 200:

```
{
  "circuit_id": "ckt_7f3a21b",
  "name": "IEEE13Nodeckt",
  "num_buses": 16,
  "num_elements": 58,
  "converged": true,
  "total_power_kw": -3466.1,
  "total_power_kvar": -1990.6,
```

```
  "buses": ["650", "rg60", "632", "633", "634", "645", "646", "670",
"671", "675", "680", "684", "611", "652", "692", "rg70"],
  "buses_phases": {
    "650": [1, 2, 3],
    "632": [1, 2, 3],
    "634": [1, 2, 3],
    "645": [2, 3],
    "646": [2, 3],
    "611": [3],
    "652": [1]
  }
}
```

11.2 Endpoints de Analisis de Voltajes

`GET /api/circuit/{circuit_id}/voltage-profile`

Retorna el perfil de voltajes en por unidad de todas las barras del estado base.

Query parameters: ninguno

Respuesta 200:

```
{
  "circuit_id": "ckt_7f3a21b",
  "voltage_profile": [
    { "bus_phase": "650.1", "voltage_pu": 1.000000 },
    { "bus_phase": "650.2", "voltage_pu": 1.000000 },
    { "bus_phase": "650.3", "voltage_pu": 1.000000 },
    { "bus_phase": "632.1", "voltage_pu": 1.021456 },
    { "bus_phase": "632.2", "voltage_pu": 1.019823 },
    { "bus_phase": "632.3", "voltage_pu": 1.020341 },
    { "bus_phase": "634.1", "voltage_pu": 0.994521 }
  ],
  "limits": {
    "lower": 0.95,
    "upper": 1.05
  },
  "violations": []
}
```

11.3 Endpoints de Perdidas del Sistema

`GET /api/circuit/{circuit_id}/losses`

Retorna la tabla de perdidas del sistema en estado base.

Respuesta 200:

```
{
  "circuit_id": "ckt_7f3a21b",
  "summary": {
    "total_losses_kw": 87.34,
    "total_losses_kvar": 42.15,
    "total_load_kw": 3466.10
  },
  "elements": [
    {
      "type": "Lines",
      "element": "650632",
      "losses_kw": 12.450,
      "losses_pct": 0.36,
      "losses_kvar": 6.230
    },
    {
      "type": "Transformers",
      "element": "Sub",
      "losses_kw": 34.120,
      "losses_pct": 0.98,
      "losses_kvar": 16.780
    }
  ]
}
```

11.4 Endpoints de Simulacion con Generacion Distribuida

`POST /api/circuit/{circuit_id}/simulate`

Ejecuta una simulacion con una GD aplicada y retorna el analisis comparativo completo.

Body (JSON):

```
{
  "bus": "634",
  "phases": [1, 2, 3],
  "type": "three_phase",
  "power_kw": 500.0
}
```

Campos del body:

Campo	Tipo	Requerido	Descripcion	Valores validos

Campo	Tipo	Requerido	Descripcion	Valores validos
`bus`	string	Si	Nombre de la barra de conexion	Barra existente en el circuito
`phases`	List[int]	Si	Fases a utilizar	Subconjunto de fases disponibles en la barra
`type`	string	Si	Tipo de conexion de la GD	`"single_phase"`, `"two_phase"`, `"three_phase"`
`power_kw`	float	Si	Potencia activa de la GD en kW	0.0 a 150,000.0

Respuesta 200:

```
{
  "circuit_id": "ckt_7f3a21b",
  "simulation_id": "sim_9b4c12d",
  "input": {
    "bus": "634",
    "phases": [1, 2, 3],
    "type": "three_phase",
    "power_kw": 500.0
  },
  "voltage_comparison": [
    {
      "bus_phase": "634.1",
      "voltage_base_pu": 0.994521,
      "voltage_with_gd_pu": 1.012430
    },
    {
      "bus_phase": "632.1",
      "voltage_base_pu": 1.021456,
      "voltage_with_gd_pu": 1.031250
    }
  ],
  "losses": {
    "base_kw": 87.34,
    "with_gd_kw": 72.81,
    "delta_kw": -14.53,
    "base_kvar": 42.15,
    "with_gd_kvar": 35.90
  },
  "violations": {
    "voltage": [],
    "current": [],
    "power": []
  },
  "converged": true
}
```

Respuesta 400 - Validacion de compatibilidad de fases:

```
{
  "error": "PHASE_INCOMPATIBILITY",
  "message": "La barra '611' solo tiene disponible la fase 3. No es
posible conectar una GD trifasica.",
  "available_phases": [3]
}
```

Respuesta 422 - Error de convergencia:

```
{
  "error": "CIRCUIT_DID_NOT_CONVERGE",
  "message": "El circuito no convergio con los parametros especificados",
  "suggestion": "Reduzca la potencia de la GD o elija una barra diferente"
}
```

11.5 Endpoints de Hosting Capacity

`POST /api/circuit/{circuit_id}/hosting-capacity`

Inicia el calculo de hosting capacity para todas las barras. Retorna un task_id para consulta asincrona.

Body (JSON):

```
{
  "max_power_kw": 1500000,
  "precision_kw": 1,
  "check_voltage": true,
  "check_current": true,
  "check_power": true
}
```

Campos del body:

Campo	Tipo	Requerido	Default	Descripcion
`max_power_kw`	float	No	1500000	Limite superior de la busqueda binaria
`precision_kw`	float	No	1	Resolucion minima de la busqueda
`check_voltage`	bool	No	true	Verificar violaciones de voltaje
`check_current`	bool	No	true	Verificar violaciones de corriente
`check_power`	bool	No	true	Verificar violaciones de potencia

Respuesta 202:

```
{
  "task_id": "task_a1b2c3d4",
  "status": "queued",
  "estimated_duration_seconds": 180,
  "poll_url": "/api/tasks/task_a1b2c3d4/status"
}
```

`GET /api/circuit/{circuit_id}/hosting-capacity/{bus}`

Retorna el hosting capacity ya calculado para una barra especifica.

Path parameters:

- `circuit_id` (string)
- `bus` (string): nombre de la barra

Respuesta 200:

```
{
  "bus": "634",
  "phases": [
    { "phase": 1, "max_gd_kw": 87500, "limiting_constraint": "voltage" },
    { "phase": 2, "max_gd_kw": 91200, "limiting_constraint": "current" },
    { "phase": 3, "max_gd_kw": 89100, "limiting_constraint": "voltage" }
  ]
}
```

`GET /api/circuit/{circuit_id}/hosting-capacity`

Retorna la tabla completa de hosting capacity para todas las barras del circuito.

Respuesta 200:

```
{
  "circuit_id": "ckt_7f3a21b",
  "results": [
    { "bus": "650", "phase": 1, "max_gd_kw": 125000,
"limiting_constraint": "voltage" },
    { "bus": "650", "phase": 2, "max_gd_kw": 118500,
"limiting_constraint": "current" },
    { "bus": "634", "phase": 1, "max_gd_kw": 87500,
"limiting_constraint": "voltage" },
    { "bus": "611", "phase": 3, "max_gd_kw": 42100,
"limiting_constraint": "power" }
```

```
    ],
    "pivot": {
      "650": { "1": 125000, "2": 118500, "3": 122300 },
      "634": { "1": 87500,  "2": 91200,  "3": 89100  },
      "611": { "3": 42100 }
    }
  }
```

11.6 Endpoints de Monitoreo de Tareas Asincronas

`GET /api/tasks/{task_id}/status`

Consulta el estado de una tarea en ejecucion.

Respuesta 200 (en progreso):

```
{
  "task_id": "task_a1b2c3d4",
  "status": "running",
  "progress_pct": 45,
  "current_step": "Calculando barra 634, fase 2",
  "buses_completed": 8,
  "buses_total": 16,
  "started_at": "2025-04-07T14:30:00Z"
}
```

Respuesta 200 (completado):

```
{
  "task_id": "task_a1b2c3d4",
  "status": "completed",
  "progress_pct": 100,
  "result_url": "/api/circuit/ckt_7f3a21b/hosting-capacity",
  "completed_at": "2025-04-07T14:35:12Z",
  "duration_seconds": 312
}
```

Respuesta 200 (error):

```
{
  "task_id": "task_a1b2c3d4",
  "status": "failed",
  "error": "CIRCUIT_ERROR",
  "message": "El motor OpenDSS encontro un error durante la simulacion",
```

```
  "failed_at": "2025-04-07T14:31:05Z"
}
```

11.7 Endpoints de Exportacion

`GET /api/circuit/{circuit_id}/export/excel`

Exporta los resultados de analisis a un archivo Excel con multiples hojas.

Query parameters:

Parametro	Tipo	Default	Descripcion
`include_voltages`	bool	true	Incluir hoja de perfil de voltajes
`include_losses`	bool	true	Incluir hoja de perdidas
`include_hosting_capacity`	bool	true	Incluir hoja de hosting capacity

Respuesta 200:

- Content-Type: `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`
- Body: archivo Excel binario
- Hojas: `Voltajes`, `Perdidas`, `Hosting_Capacity`, `Violaciones`

`GET /api/circuit/{circuit_id}/export/json`

Exporta todos los resultados en formato JSON para integracion con otros sistemas.

Respuesta 200:

```
{
  "circuit_id": "ckt_7f3a21b",
  "exported_at": "2025-04-07T14:40:00Z",
  "circuit_info": { ... },
  "voltage_profile": [ ... ],
  "losses": { ... },
  "hosting_capacity": [ ... ]
}
```

11.8 Resumen de Endpoints

Metodo	Endpoint	Descripcion
POST	`/api/circuit/upload`	Subir y compilar circuito DSS

Metodo	Endpoint	Descripcion
GET	`/api/circuit/{id}/info`	Informacion general del circuito
GET	`/api/circuit/{id}/voltage-profile`	Perfil de voltajes base
GET	`/api/circuit/{id}/losses`	Tabla de perdidas base
POST	`/api/circuit/{id}/simulate`	Simular con GD aplicada
POST	`/api/circuit/{id}/hosting-capacity`	Iniciar calculo de hosting capacity (asincrono)
GET	`/api/circuit/{id}/hosting-capacity`	Obtener tabla completa de hosting capacity
GET	`/api/circuit/{id}/hosting-capacity/{bus}`	Hosting capacity por barra
GET	`/api/tasks/{task_id}/status`	Estado de tarea asincrona
GET	`/api/circuit/{id}/export/excel`	Exportar resultados a Excel
GET	`/api/circuit/{id}/export/json`	Exportar resultados a JSON

Apendice A: Circuito IEEE 13 Nodos

El circuito de referencia tiene las siguientes caracteristicas:

Parametro	Valor
Nombre	IEEE13Nodeckt
Tension de red primaria	115 kV
Tension de red secundaria	4.16 kV
Potencia del transformador de subestacion	5 MVA
Corriente de cortocircuito trifasica	20,000 MVA
Numero de barras	16
Numero de lineas	10
Numero de cargas	11
Potencia total de carga aproximada	3,466 kW / 1,991 kvar
Capacitores	4 (total 1,200 kvar)
Reguladores de tension	6 (monofasicos)
Rango de voltaje aceptable	0.95 a 1.05 PU

Barras y Fases Disponibles

Barra	Fases
650	1, 2, 3
RG60	1, 2, 3
RG70	1, 2, 3
632	1, 2, 3
633	1, 2, 3
634	1, 2, 3
645	2, 3
646	2, 3
670	1, 2, 3
671	1, 2, 3
675	1, 2, 3
680	1, 2, 3
684	1, 3
611	3
652	1
692	1, 2, 3

Apendice B: Codigos de Error Recomendados

Codigo	Descripcion
`INVALID_FILE_FORMAT`	El archivo subido no tiene formato OpenDSS valido
`CIRCUIT_NOT_FOUND`	No se encontro el circuito con el ID indicado
`CIRCUIT_NOT_CONVERGED`	El motor OpenDSS no logro convergencia
`PHASE_INCOMPATIBILITY`	Las fases solicitadas no estan disponibles en la barra
`POWER_OUT_OF_RANGE`	La potencia de la GD esta fuera del rango permitido
`TASK_NOT_FOUND`	No se encontro la tarea con el ID indicado
`TASK_STILL_RUNNING`	La tarea no ha finalizado; usar el endpoint de status
`ENGINE_ERROR`	Error interno del motor OpenDSS
`CIRCUIT_EXPIRED`	El circuito ha expirado; volver a cargar el archivo